

Untitled13

April 1, 2026

```
[1]: !pip install nltk scikit-learn
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: nltk in ~/.local/lib/python3.10/site-packages
(3.8.1)
Requirement already satisfied: scikit-learn in ~/.local/lib/python3.10/site-
packages (1.3.2)
Requirement already satisfied: joblib in ~/.local/lib/python3.10/site-packages
(from nltk) (1.3.2)
Requirement already satisfied: tqdm in ~/.local/lib/python3.10/site-packages
(from nltk) (4.66.2)
Requirement already satisfied: regex>=2021.8.3 in ~/.local/lib/python3.10/site-
packages (from nltk) (2023.12.25)
Requirement already satisfied: click in /usr/lib/python3/dist-packages (from
nltk) (8.0.3)
Requirement already satisfied: scipy>=1.5.0 in /usr/lib/python3/dist-packages
(from scikit-learn) (1.8.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in
~/.local/lib/python3.10/site-packages (from scikit-learn) (3.2.0)
Requirement already satisfied: numpy<2.0,>=1.17.3 in
~/.local/lib/python3.10/site-packages (from scikit-learn) (1.24.4)
```

```
[4]: import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
from sklearn.feature_extraction.text import TfidfVectorizer

# Download required datasets
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt to /home/dypcoe-
[nltk_data]      admin/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

```
[nltk_data] Downloading package stopwords to /home/dypcoe-
[nltk_data]   admin/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   /home/dypcoe-admin/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
[nltk_data] Downloading package wordnet to /home/dypcoe-
[nltk_data]   admin/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

[4]: True

```
[5]: text = """Text analytics is the process of deriving meaningful information
from natural language text. It involves tokenization, tagging, and_
normalization."""
```

```
[6]: tokens = word_tokenize(text)
print("Tokens:", tokens)
```

```
Tokens: ['Text', 'analytics', 'is', 'the', 'process', 'of', 'deriving',
'meaningful', 'information', 'from', 'natural', 'language', 'text', '.', 'It',
'involves', 'tokenization', ',', 'tagging', ',', 'and', 'normalization', '.']
```

```
[7]: pos_tags = pos_tag(tokens)
print("POS Tags:", pos_tags)
```

```
POS Tags: [('Text', 'NN'), ('analytics', 'NNS'), ('is', 'VBZ'), ('the', 'DT'),
('process', 'NN'), ('of', 'IN'), ('deriving', 'VBG'), ('meaningful', 'JJ'),
('information', 'NN'), ('from', 'IN'), ('natural', 'JJ'), ('language', 'NN'),
('text', 'NN'), ('.', '.'), ('It', 'PRP'), ('involves', 'VBZ'), ('tokenization',
'NN'), (',', ','), ('tagging', 'NN'), (',', ','), ('and', 'CC'),
('normalization', 'NN'), ('.', '.')]

```

```
[8]: stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]

print("After Stopword Removal:", filtered_tokens)
```

```
After Stopword Removal: ['Text', 'analytics', 'process', 'deriving',
'meaningful', 'information', 'natural', 'language', 'text', '.', 'involves',
'tokenization', ',', 'tagging', ',', 'normalization', '.']
```

```
[9]: ps = PorterStemmer()
stemmed_words = [ps.stem(word) for word in filtered_tokens]

print("Stemmed Words:", stemmed_words)
```

```
Stemmed Words: ['text', 'analyt', 'process', 'deriv', 'meaning', 'inform',  
'natur', 'languag', 'text', '.', 'involv', 'token', ',', 'tag', ',', 'normal',  
'..']
```

```
[10]: lemmatizer = WordNetLemmatizer()  
      lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_tokens]  
  
      print("Lemmatized Words:", lemmatized_words)
```

```
Lemmatized Words: ['Text', 'analytics', 'process', 'deriving', 'meaningful',  
'information', 'natural', 'language', 'text', '.', 'involves', 'tokenization',  
,', 'tagging', ',', 'normalization', '..']
```

```
[11]: from collections import Counter  
  
      tf = Counter(filtered_tokens)  
      print("Term Frequency:", tf)
```

```
Term Frequency: Counter({'.': 2, ',': 2, 'Text': 1, 'analytics': 1, 'process':  
1, 'deriving': 1, 'meaningful': 1, 'information': 1, 'natural': 1, 'language':  
1, 'text': 1, 'involves': 1, 'tokenization': 1, 'tagging': 1, 'normalization':  
1})
```

```
[12]: documents = [text]  
  
      vectorizer = TfidfVectorizer()  
      tfidf_matrix = vectorizer.fit_transform(documents)  
  
      print("Feature Names:", vectorizer.get_feature_names_out())  
      print("TF-IDF Matrix:\n", tfidf_matrix.toarray())
```

```
Feature Names: ['analytics' 'and' 'deriving' 'from' 'information' 'involves'  
'is' 'it'  
'language' 'meaningful' 'natural' 'normalization' 'of' 'process'  
'tagging' 'text' 'the' 'tokenization']  
TF-IDF Matrix:  
[[0.21821789 0.21821789 0.21821789 0.21821789 0.21821789 0.21821789  
 0.21821789 0.21821789 0.21821789 0.21821789 0.21821789 0.21821789  
 0.21821789 0.21821789 0.21821789 0.43643578 0.21821789 0.21821789]]
```